The visibility of class constants is controlled in the same way as other members, with `public`, `protected`, and so on. It is not possible to declare a type for a constant in PHP <= 8.2, but support for typed class constants was added in PHP 8.3. By convention (and many coding standards), class names are usually all uppercase with words separated by underscores; otherwise, they're subject to the usual variable naming rules.

Accessing a class constant also uses the scope resolution operator:

```
print QuoteBot::FILM;
print $quoteBot::FILM;
```

This is why we needed the dollar symbol for static property access: so that property access is syntactically distinct from accessing class constants.

Private Constructors

Here's an interesting technique that makes use of class context code to control how objects are created.

It is possible to declare a constructor as `private` in PHP. This has the obvious effect of hiding the `__construct()` method; it is no longer callable from outside the class. This is what happens when you try to instantiate a class with a `private` constructor:

```
class SingleQuoteBot
{
    protected array $quoteStash = [
        "Obviously, you're not a golfer." . PHP_EOL,
        "That's just, like, your opinion, man." . PHP_EOL,
        "Forget it, Donny You're out of your element." . PHP_EOL,
    ];

    private function __construct()
    {
    }

    public function getQuote(): string
    {
        return $this->quoteStash[rand(0,2)];
    }
}
```